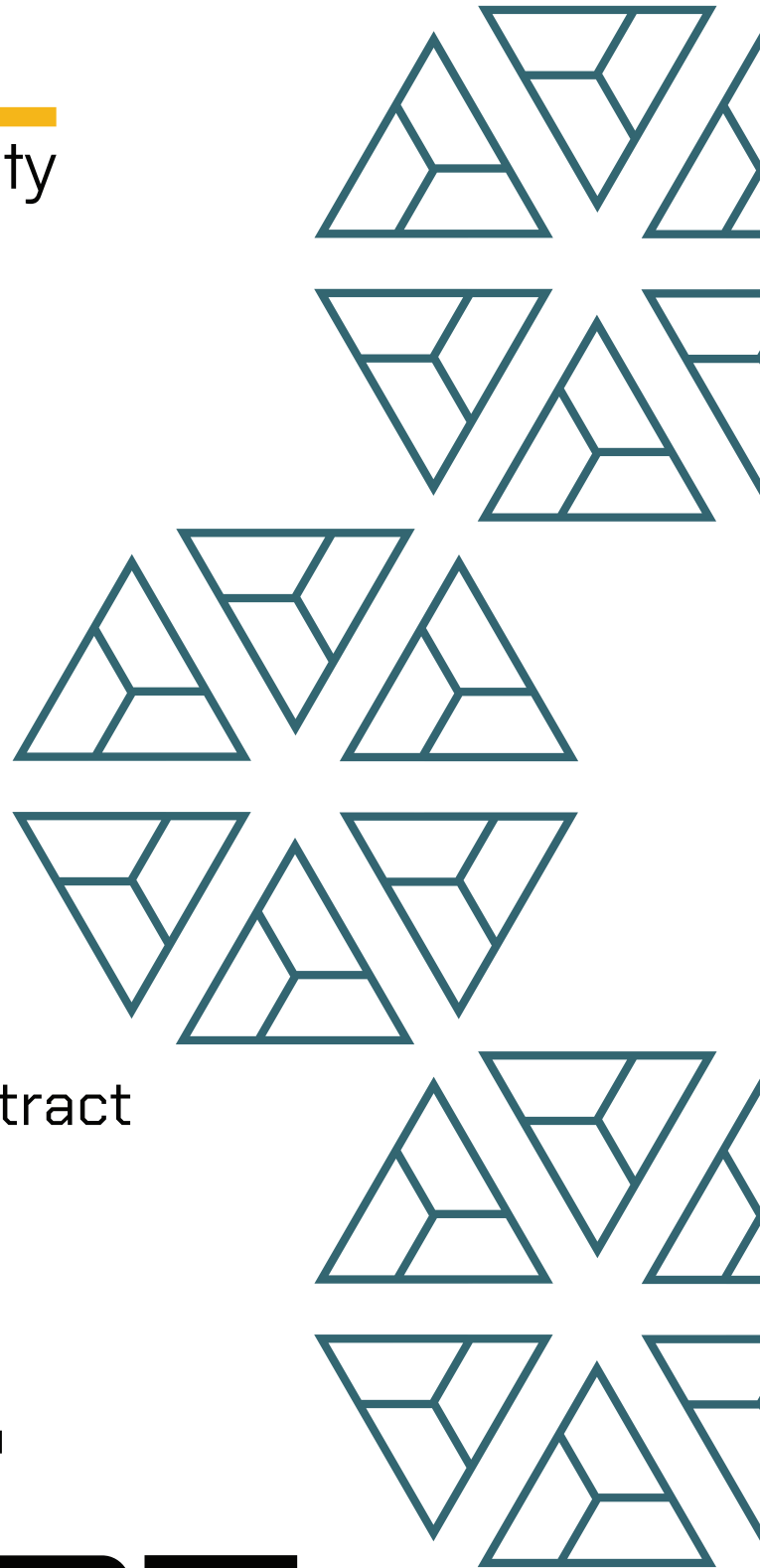




BAIL
security



Dowsure

Intermediate Deposit Contract

FINAL REPORT

March '2026

Disclaimer:

Security assessment projects are time-boxed and often reliant on information that may be provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

The content of this assessment is not an investment. The information provided in this report is for general informational purposes only and is not intended as investment, legal, financial, regulatory, or tax advice. The report is based on a limited review of the materials and documentation provided at the time of the audit, and the audit results may not be complete or identify all possible vulnerabilities or issues. The audit is provided on an "as-is," "where-is," and "as-available" basis, and the use of blockchain technology is subject to unknown risks and flaws.

The audit does not constitute an endorsement of any particular project or team, and we make no warranties, expressed or implied, regarding the accuracy, reliability, completeness, or availability of the report, its content, or any associated services or products. We disclaim all warranties, including the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

We assume no responsibility for any product or service advertised or offered by a third party through the report, any open-source or third-party software, code, libraries, materials, or information linked to, called by, referenced by, or accessible through the report, its content, and the related services and products. We will not be liable for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract.

The contract owner is responsible for making their own decisions based on the audit report and should seek additional professional advice if needed. The audit firm or individual assumes no liability for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract. The contract owner agrees to indemnify and hold harmless the audit firm or individual from any and all claims, damages, expenses, or liabilities arising from the use or reliance on the audit report or the smart contract.

By engaging in a smart contract audit, the contract owner acknowledges and agrees to the terms of this disclaimer.

1. Project Details

Important:

Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Dowsure - Intermediate Deposit Contract - Audit Report
Website	---
Language	Solidity
Methods	Manual Analysis
Github repository	---
Resolution 1	---

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no changes made)	Failed resolution	Open
High	0					
Medium	1					
Low	4					
Informational	9					
Governance	3					
Total	17					

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

This audit has been conducted on placeholder contracts deployed to only accept user funds. The contracts will later be replaced with the actual smart contracts implementing the full functionality of the protocol once its ongoing audit has been concluded. The issues listed below are only valid for these placeholder contracts, and not for the core protocol.

ProtocolCfg

The `ProtocolCfg` contract is a protocol-level configuration contract that maintains a whitelist of supported ERC-20 underlying assets. It uses OpenZeppelin's `AccessControl` to restrict asset management to addresses with the `PROTOCOL_OWNER_ROLE`, who can add or remove tokens from the supported set. Other contracts in the protocol (e.g., `Vault`, `VaultFactory`) can query `isSupportedUnderlyingAsset` to enforce that only approved tokens are used.

Privileged Functions

- `addUnderlyingAsset`
- `removeUnderlyingAsset`

Core Invariants:

INV 1: Only the `PROTOCOL_OWNER_ROLE` role is allowed to add/remove assets.

Issue_01	Consider using named mappings to improve code readability
Severity	Informational
Description	The <code>protocolCfg</code> contract declares unnamed key-value fields of mapping <code>isSupportedUnderlyingAsset</code>. <pre>mapping(IERC20 => bool) public isSupportedUnderlyingAsset;</pre> This issue was identified in the intermediate contracts deployed to hold protocol funds. This issue does not relate to the core protocol audit.
Recommendations	Consider adding names for the key-value fields. For example: <pre>mapping(IERC20 asset => bool supported) public isSupportedUnderlyingAsset;</pre>
Comments / Resolution	

VaultFactory

The `VaultFactory` contract allows addresses with the `POOL_OPERATOR_ROLE` to deploy new `Vault` instances with specified parameters (capacity, staking duration, deadline, and accepted assets). It holds a reference to the current `ProtocolCfg` address, updatable by the `OWNER_ROLE`, and validates that all requested underlying assets are protocol-approved before creating each vault.

Privileged Functions

- `setProtocolCfg`
- `createVault`

Core Invariants:

INV 1: Only `POOL_OPERATOR_ROLE` roles can deploy vaults.

Issue_02	<code>VaultFactory</code> inherits <code>ReentrancyGuard</code> , but it is never used
Severity	Informational
Description	The <code>VaultFactory</code> contract inherits <code>ReentrancyGuard</code> but never uses a <code>nonReentrant</code> modifier or any of its functions. This issue was identified in the intermediate contracts deployed to hold protocol funds. This issue does not relate to the core protocol audit.
Recommendations	Consider not inheriting the <code>ReentrancyGuard</code> contract if it is not intended to be used.
Comments / Resolution	

Issue_03	Use custom errors instead of string error messages to save gas
Severity	Informational
Description	The VaultFactory contract uses string error messages throughout the contract. Custom errors are cheaper than require statements with strings since only 4 bytes need to be stored in memory. For string messages, Solidity has to store in memory and revert with at least 64 bytes. <pre><i>require(_owner != address[0] && _protocolCfg != address[0], "Invalid zero address");</i></pre> Similar instances exist in the Vault and protocolCfg contracts. This issue was identified in the intermediate contracts deployed to hold protocol funds. This issue does not relate to the core protocol audit.
Recommendations	Consider using custom errors
Comments / Resolution	

Issue_04	Emit event early to save gas
Severity	Informational
Description	Function <code>setProtocolCfg</code> stores the old <code>protocolCfg</code> value in a memory variable, which is only intended to be used in the event <code>ProtocolCfgUpdated</code> emission. However, creating this memory variable is not required if the event is emitted early. <pre>function setProtocolCfg(address _protocolCfg) external { require(hasRole[OWNER_ROLE, msg.sender], "Not owner"); require(_protocolCfg != address(0), "Invalid zero address"); address oldProtocolCfg = protocolCfg; protocolCfg = _protocolCfg; emit ProtocolCfgUpdated(protocolCfg, oldProtocolCfg, msg.sender); }</pre> This issue was identified in the intermediate contracts deployed to hold protocol funds. This issue does not relate to the core protocol audit.
Recommendations	Consider moving the event emission before the state change, so that the value does not need to be cached.
Comments / Resolution	

Vault

The **Vault** contract is an ERC-20 staking vault that accepts deposits of whitelisted underlying assets (validated against **ProtocolCfg**) up to a configurable capacity and deadline. Deposits are normalized to 18 decimals for uniform accounting. Withdrawals follow a two-step request-and-review flow: a **WITHDRAW_REQUEST_ROLE** holder creates a withdrawal request, and a **WITHDRAW_REVIEWER_ROLE** holder approves or rejects it, providing segregation of duties over outflows.

Privileged Functions

- `addUnderlyingAsset`
- `setDepositDeadline`
- `roleRequestWithdraw`
- `reviewRoleWithdrawRequest`

Core Invariants:

INV 1: Funds can only be deposited up to the **maxCapacity** cap.

INV 2: Deposits can only be made before the **depositDeadline** timestamp.

INV 3: Withdrawals can only be carried out by **WITHDRAW_REQUEST_ROLE** and **WITHDRAW_REVIEWER_ROLE** role holders.

Issue_05	All roles can be renounced to trap user funds
Severity	Governance
Description	The Vault contract has four critical roles: <code>DEFAULT_ADMIN_ROLE</code>, <code>ADMIN_ROLE</code>, <code>WITHDRAW_REQUEST_ROLE</code>, <code>WITHDRAW_REVIEWER_ROLE</code>. If these roles are renounced by the respective privileged addresses (in case of compromise) using the function <code>renounceRole</code>, there would be no way to retrieve user funds. This issue was identified in the intermediate contracts deployed to hold protocol funds. This issue does not relate to the core protocol audit.
Recommendations	Consider disallowing roles from being renounced.
Comments / Resolution	

Issue_06	Protocol dependent on contract upgrade procedure
Severity	Governance
Description	Since the current contracts are only placeholders that will be upgraded later, the performance of the core protocol ultimately depends on whether the upgrade procedure has been carried out properly, preserving the state of the protocol and user balances. This issue was identified in the intermediate contracts deployed to hold protocol funds. This issue does not relate to the core protocol audit.
Recommendations	Contract upgrade must carefully preserve the state, not adding extra storage variables in the middle, changing the slot order.
Comments / Resolution	

Issue_07	Governance can drain protocol
Severity	Governance
Description	The current withdrawal mechanism is not user-controlled and instead entirely dependent on the Governance. Thus, any private-key leaks can drain the protocol in the current state, which might be patched in a future upgrade. This issue was identified in the intermediate contracts deployed to hold protocol funds. This issue does not relate to the core protocol audit.
Recommendations	Consider upgrading the contract as soon as possible, to prevent such attack surface.
Comments / Resolution	

Issue_08	Withdrawals don't update the balance state
Severity	Medium
Description	During deposits, <code>userBalance[msg.sender][asset]</code> and <code>totalDeposited</code> are updated to keep track of balances. However, in the withdrawal mechanism, these values are not decreased again. So, users can technically withdraw more than their deposit. Thus, the withdrawal mechanism basically has no contract-based checks to prevent over-withdrawing, and completely depends on the admin tracking all deposits and withdrawals of a user and making sure they don't approve over-withdrawals. Also, if the contract is upgraded to a newer version, the <code>userBalance[msg.sender][asset]</code> can still retain values that have already been withdrawn. This issue was identified in the intermediate contracts deployed to hold protocol funds. This issue does not relate to the core protocol audit.
Recommendations	Consider updating the <code>userBalance</code> and <code>totalDeposited</code> states during withdrawals.
Comments / Resolution	

Issue_09	No way to remove a supported asset from the vault
Severity	Low
Description	The vault implements an <code>addUnderlyingAsset</code> function to add supported assets. However, there is no way to remove any supported assets from this. Thus, if an asset is deprecated, it will still always be supported by the vault. This issue was identified in the intermediate contracts deployed to hold protocol funds. This issue does not relate to the core protocol audit.
Recommendations	Consider acknowledging this behaviour since these are temporary placeholder contracts. The main contracts should allow depreciating assets.
Comments / Resolution	

Issue_10	<code>roleRequestWithdraw</code> balance check can be violated
Severity	Low
Description	In the <code>roleRequestWithdraw</code> function, there exists a check to make sure the withdrawals do not exceed the contract balance. <pre><i>require(asset.balanceOf(address(this)) >= amount</i></pre> However, the <code>roleRequestWithdraw</code> doesn't really move any funds and instead just queues withdrawals. So even if the contract only has 1000 tokens, it will allow <code>roleRequestWithdraw</code> to be called for 1000 tokens multiple times, creating an obligation exceeding the actual contract balance, violating the purpose of this check. This issue was identified in the intermediate contracts deployed to hold protocol funds. This issue does not relate to the core protocol audit.
Recommendations	In the <code>roleRequestWithdraw</code> function, consider caching the withdrawal request amounts so that the balance check can make sure the total withdrawal obligations don't exceed the balance.
Comments / Resolution	

Issue_11	The asset is checked twice for support in the same transaction
Severity	Low
Description	When calling <code>createVault</code>, there is a loop that loops through the <code>_underlyingAssets</code> and checks that it is a supported asset by calling <code>isSupportedUnderlyingAsset</code> on <code>protocolCfg</code>. After this is done, the contract creates a new vault passing the <code>_underlyingAssets</code> to the constructor. Again, the constructor iterates through <code>_underlyingAssets</code> and checks for support by calling the <code>protocolCfg</code> contract, which is redundant and wastes considerable gas when there are many assets that are to be supported. This issue was identified in the intermediate contracts deployed to hold protocol funds. This issue does not relate to the core protocol audit.
Recommendations	Consider removing this check from the factory, as it is redundant and wastes gas.
Comments / Resolution	

Issue_12	Existing vaults utilize stale <code>protocolCfg</code> address
Severity	Low
Description	When a <code>Vault</code> is deployed through the <code>VaultFactory</code>, the <code>protocolCfg</code> address remains immutable in the Vault's storage. However, the <code>protocolCfg</code> address can be updated by the owner on the <code>VaultFactory</code> using <code>setProtocolCfg</code>. <pre><i>Vault vault = new Vault(protocolCfg, _defaultAdmin, _maxCapacity, _stakingDays, _depositDeadline, _underlyingAssets);</i></pre> If the <code>protocolCfg</code> is updated, all existing vaults continue using the stale address, and cannot incorporate this new address. This issue was identified in the intermediate contracts deployed to hold protocol funds. This issue does not relate to the core protocol audit.
Recommendations	Consider dynamically retrieving the <code>protocolCfg</code> address from the factory, or acknowledge the issue.
Comments / Resolution	

Issue_13	<code>maxCapacity</code> needs to be defined in 18 decimals
Severity	Informational
Description	The <code>maxCapacity</code> value is compared to the normalized <code>totalDeposited</code> amount and so needs to be defined in 18 decimals and not the decimals of the asset token. This must be kept in mind when deploying and setting up the contracts. This issue was identified in the intermediate contracts deployed to hold protocol funds. This issue does not relate to the core protocol audit.
Recommendations	Initialize <code>totalDeposited</code> in 18 decimals.
Comments / Resolution	

Issue_14	ADMIN_ROLE is not granted in the constructor
Severity	Informational
Description	The constructor only grants the DEFAULT_ADMIN_ROLE. However, most contract setting operations require the ADMIN_ROLE, which is not granted here. This requires the DEFAULT_ADMIN_ROLE to grant the ADMIN_ROLE to some address in a separate transaction. This issue was identified in the intermediate contracts deployed to hold protocol funds. This issue does not relate to the core protocol audit.
Recommendations	Consider granting the ADMIN_ROLE in the constructor itself, or make sure the deployment script grants the ADMIN_ROLE to some address.
Comments / Resolution	

Issue_15	Unused stakingDays variable
Severity	Informational
Description	The stakingDays variable is initialized on deployment but never utilized in the Vault contract. <pre><i>uint256 public stakingDays;</i></pre> This issue was identified in the intermediate contracts deployed to hold protocol funds. This issue does not relate to the core protocol audit.
Recommendations	Consider removing the variable or implementing logic to utilize it.
Comments / Resolution	

Issue_16	Remove redundant require statement
Severity	Informational
Description	The require statement below is not required in the constructor of the <code>Vault</code> contract. <pre><i>require(!_protocolCfg != address[0], "Address cannot be zero");</i></pre> This is because the <code>VaultFactory</code> ensures the <code>protocolCfg</code> passed is never the zero address, as seen below. Snippet from <code>VaultFactory</code> constructor: <pre><i>require(!_owner != address[0] && !_protocolCfg != address[0], "Invalid zero address");</i></pre> Snippet from <code>VaultFactory.setProtocolCfg</code>: <pre><i>require(!_protocolCfg != address[0], "Invalid zero address");</i></pre> This issue was identified in the intermediate contracts deployed to hold protocol funds. This issue does not relate to the core protocol audit.
Recommendations	Consider removing the require statement.
Comments / Resolution	

Issue_17	Emit events on state changes
Severity	Informational
Description	The <code>setDepositDeadline</code> function does not emit any event when the <code>depositDeadline</code> is increased or decreased from the original deadline. This could mislead off-chain monitoring that relies on accurate deposit deadline data. <pre>function setDepositDeadline(uint256 deadline) external onlyRole[ADMIN_ROLE] { require(deadline > block.timestamp, "Deposit deadline must be in the future"); depositDeadline = deadline; }</pre> This issue was identified in the intermediate contracts deployed to hold protocol funds. This issue does not relate to the core protocol audit.
Recommendations	Consider emitting an event at the end of the function.
Comments / Resolution	